



Mapa de repositorios y guía de entrega

Qué va en cada repo, cómo se conectan y cómo
enviar la preselección del domingo 4/10 a las 16:00

Complemento del plan completo de Salua (secciones 2, 6 y 8)

3 de octubre de 2026 · Superteam Argentina · Colosseum Crypto World's Fair

1. Cómo está organizado el proyecto

El plan v2 del equipo define **cuatro repositorios** en la cuenta **mpeloc**. Uno es la “puerta de entrada” con documentación y demo, y los otros tres son de código. Los tres de código se clonan dentro de la carpeta del primero, que los ignora en su `.gitignore`. Se mantienen los nombres actuales hasta decidir el renombrado a Salua (sección 6).

Repositorio	Para qué sirve	Tecnología	Dueño	Dónde se publica
hcd (salua)	Puerta de entrada: README, documentación, demo navegable , pitch y materiales de marca	Markdown, HTML estático	Rodrigo (docs, pitch) y Franco (README, decisiones)	Vercel, carpeta demo
hcd_api (salua_api)	Programa de Solana, backend y servicio de llaves	Rust + Anchor, NestJS, supabase-js, Zod	Misael (programa), Matías (API), Franco (llaves y tx)	Programa en devnet. API en Railway o Render
hcd_app (salua_app)	Aplicación: paciente, médico, clínica y admin	Next.js, React, TypeScript, Tailwind, shadcn/ui, Privy, WebCrypto	Maximiliano, con Franco en cifrado e integración	Vercel
hcd_landing (salua_landing)	Página de presentación pública	Next.js estático	Rodrigo, con ayuda de Maximiliano	Vercel

Qué enlace va en cada formulario del domingo: como “repositorio” se pone **hcd**, porque su README lleva a todo lo demás. Como “demo” se pone la dirección pública de la carpeta demo. Los repos de código (hcd_api, hcd_app) se mencionan en el README y en “Algo más”.

Qué pasa con el prototipo navegable

El plan v2 dice que ya existe un prototipo sin backend con las vistas de paciente y médicos. Para el domingo conviene que viva en hcd/demo/ como archivos estáticos, porque se publica en minutos y no depende del backend ni de la cadena. Cuando hcd_app tenga el recorrido real, el prototipo queda como respaldo del pitch.

2. Detalle de cada repositorio

hcd · documentación, demo y pitch

hcd/	
├── README.md	Portada: qué es, enlaces, cómo correr, equipo, límites
├── demo/	Prototipo navegable (HTML estático, sin backend)
│ └── index.html	
├── pitch/	
│ ├── deck.pdf	Deck de 8 diapositivas
│ ├── guion.md	Guion de 4:30 y quién habla
│ └── video.md	Enlace al video (YouTube no listado)
├── assets/	Logo (PNG y SVG), paleta, capturas
├── docs/	
│ ├── proyecto/	plan.md, stack.md, decisiones.md
│ └── legal/	Notas legales orientativas
├── .github/	Hooks compartidos por todos los repos
└── .gitignore	Ignora hcd_api/, hcd_app/ y hcd_landing/

- **Regla del README:** alguien que nunca vio el proyecto tiene que entender en un minuto qué es, ver el demo y encontrar cada repo. Hay una plantilla en la sección 6.
- **Código y commits en inglés, documentación en español**, como define el plan v2.

hcd_api · programa de Solana, backend y servicio de llaves

```
hcd_api/
├── programs/hcd/src/      Programa Anchor (Rust)
│   ├── lib.rs · errors.rs · events.rs
│   ├── state/            Config, Provider, PatientProfile, Record, AccessGrant
│   └── instructions/     issue_record, dispute_record, void_record,
│                           grant_access, revoke_access, log_access, ...
├── tests/                Tests de Anchor: positivos y negativos
├── idl/                  hcd.json: IDL publicado, del que otros generan su cliente
├── supabase/migrations/  SQL: tablas, RLS y rol app_user
├── src/
│   ├── auth/             Guard del token de Supabase Auth
│   ├── organizations/    Clínicas, médicos y verificación
│   ├── records/          URLs firmadas, metadatos, llave envuelta
│   ├── access/           Solicitudes de acceso y notificaciones
│   ├── keys/             Servicio de llaves: /keys/release
│   ├── tx/               Arma y co-firma transacciones, fee payer
│   ├── indexer/          Escucha los eventos del programa
│   └── common/           Esquemas Zod, configuración, errores, logs
└── Anchor.toml · package.json · .env.example · README.md
```

Carpeta	Responsable	Entregable mínimo para el domingo 11/10
programs/ y tests/	Misael	Instrucciones principales (issue, dispute, void, grant, revoke y log_access) con tests negativos pasando y desplegadas en devnet
idl/	Misael	IDL v0 publicado el lunes 5 a la noche y actualizado con cada cambio
supabase/ y src/ (auth, organizations, records, access, indexer)	Matías	Login, tablas con RLS, subida de archivos cifrados, solicitudes y auditoría
src/keys/ y src/tx/	Franco	Entrega de llaves con verificación en Solana, y transacciones armadas y co-firmadas

hcd_app · aplicación

```
hcd_app/
├── app/
│   ├── (auth)/login/
│   ├── (paciente)/      inicio, estudios, qr, accesos, linea-de-tiempo
│   ├── (medico)/        escanear, cargar, solicitar, visor/[recordId]
│   ├── (clinica)/       avalar-medicos
│   └── (admin)/         verificar
├── components/          UI con shadcn/ui, visor, marca de agua, QR
├── lib/
│   ├── hcd-client/      Cliente generado con Codama desde idl/hcd.json
│   ├── crypto/          AES-256-GCM, SHA-256 y utilidades
│   ├── schemas/         Esquemas Zod copiados de hcd_api
│   └── supabase.ts · privy.ts · api.ts
├── public/              Logo, iconos de la PWA, manifest
└── next.config.ts · package.json · .env.example · README.md
```

- **Maximiliano:** pantallas, QR, listas, aprobaciones, visor y línea de tiempo, con la marca Salua.
- **Franco:** lib/crypto/ (cifrado y verificación del hash) y la integración con la API y la cadena.

hcd_landing · página pública

```
hcd_landing/
├── app/page.tsx         Hero, problema, cómo funciona, seguridad, equipo, enlace al demo
├── components/
├── public/              Logo, imágenes, imagen para compartir en redes
└── next.config.ts       output: 'export' (sitio estático)
```

No es necesaria para la preselección del domingo. Se hace después del recorrido completo (jueves 8 y sábado 10).

3. Cómo se conectan los repositorios

Conexión	Cómo funciona	Quién la cuida
Programa a API y app	El IDL de hcd_api/idl/ es la fuente. Cada repo genera su cliente con Codama desde ese archivo. Si el IDL cambia, hay que regenerar	Misael publica, Matías y Maximiliano regeneran
App a API	La app llama a la API por HTTPS con el token de Supabase Auth en la cabecera. Los esquemas Zod se copian a mano de hcd_api a hcd_app	Matías y Maximiliano
Transacciones	La API arma la transacción, el usuario firma con su wallet de Privy y la API verifica byte a byte antes de co-firmar como fee payer	Franco
Supabase	Un solo proyecto compartido. Solo la API escribe. Para leer, la API crea un cliente por solicitud con el token del usuario, así RLS decide qué filas ve	Matías
Privy	El ID de aplicación es público y va en la app. El secreto va solo en la API	Maximiliano y Matías
Documentación	hcd enlaza a todo y guarda el plan y las decisiones	Rodrigo y Franco

Un estudio de punta a punta: qué toca cada parte

Paso	hcd_app	hcd_api (src)	Programa	Supabase
El médico escanea el QR del paciente	Escáner y validación del código	Valida el código de 2 minutos	—	Código vigente
Carga del estudio	Cifra con AES-256-GCM y calcula el hash	URL firmada y envoltura de la llave (keys)	—	Archivo cifrado y llave envuelta
Firma de la emisión	Pide la transacción y la firma con Privy	Arma, verifica y co-firma (tx)	issue_record: Record Active	Metadatos del Record
El paciente aprueba un acceso	Pantalla de aprobación	Arma la transacción	grant_access: AccessGrant	Solicitud marcada como aprobada
El médico lee	Pide la llave, verifica el hash, descifra y muestra	keys comprueba Record y Grant en Solana	log_access suma access_count	Registro de la entrega
Línea de tiempo	Muestra los accesos	El indexer escucha los eventos	Eventos emitidos	Tabla de auditoría

Variables de entorno y secretos

Variable	Dónde vive	Para qué
SUPABASE_URL, SUPABASE_ANON_KEY	API y app (públicas)	Conexión a Supabase y login
DATABASE_URL (rol app_user)	Solo API	Consultas con RLS. No usar la clave de servicio para leer
SUPABASE_SERVICE_ROLE	Solo API, uso mínimo	Operaciones de storage. Saltea RLS, por eso se limita
FEE_PAYER_SECRET	Solo API	Paga las comisiones de red. Billetera con poco saldo
KEY_SERVICE_SECRET	Solo API	Firma log_access. Distinta del fee payer
MASTER_KEY	Solo API	Llave maestra para envolver las llaves de los estudios
PRIVY_APP_SECRET	Solo API	Verificación de sesiones y wallets
NEXT_PUBLIC_PRIVY_APP_ID, NEXT_PUBLIC_PROGRAM_ID, NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SOLANA_RPC_URL	App (públicas)	Configuración del navegador. Nunca van secretos acá

Regla: los secretos viven en las variables de entorno de Railway, Render o Vercel y en un archivo .env local ignorado por Git. En el repositorio solo hay un .env.example con los nombres, sin valores.

4. Cómo trabajar dentro de los repositorios

Tema	Regla
Ramas	main siempre funciona y nadie sube directo. Ramas por tarea: feat/, fix/, docs/
Commits	En inglés y cortos, por ejemplo feat: add dispute_record instruction
Pull requests	Una revisión como mínimo. El programa de Anchor lo revisa alguien distinto a Misael. Merge con squash
Issues	Uno por tarea, con responsable y fecha. Etiquetas: programa, api, app, docs, bloqueado
Dos personas en hcd_api	Cada una en su carpeta (programs/ y tests/ para Misael, src/ para Matías y Franco) para no chocar
Cambios de contrato	Si cambia el IDL o un esquema Zod, avisar en el grupo antes de mergear y regenerar los clientes
Listo significa	Funciona en devnet, tiene test o prueba manual anotada, está revisado y mergeado

Permisos y visibilidad en GitHub

- Solo el dueño de la cuenta **pmpelec** (o un administrador del repositorio) puede renombrar o cambiar la visibilidad. Hay que pedirselo hoy.
- Todos los integrantes deben figurar como colaboradores con permiso de escritura en los cuatro repos.
- Para la preselección, **hcd** tiene que ser público. Los repos de código pueden seguir privados hasta que tengan un README presentable, pero antes del 12/10 también deben ser públicos o estar compartidos con quien pida el hackathon.

5. Preparativos de hoy, sábado a la noche

#	Tarea	Quién	Listo cuando
1	Cada integrante se anota en el Luma de Argentina y se registra en Colosseum Arena, con el mismo email en todos los pasos	Todos	Cinco confirmaciones
2	Armar el equipo "Salua" en Arena e invitar a los otros cuatro. Seguir lo que pida la plataforma	Franco	Los cinco aparecen en el equipo
3	Abrir los formularios de los pasos 3 y 4 y copiar todos sus campos a una lista	Franco y Rodrigo	Lista de campos en el grupo
4	Publicar el demo en Vercel desde hcd/demo (comandos abajo)	Maximiliano	Enlace abierto en una ventana de incógnito
5	Aplicar nombre, logo y colores de Salua al demo. Que no quede "HCD" visible	Maximiliano	Demo con marca Salua
6	Escribir el README de hcd (plantilla abajo)	Rodrigo y Franco	README con enlaces que funcionan
7	Armar el deck de 8 diapositivas y exportarlo a PDF	Rodrigo	pitch/deck.pdf subido
8	Grabar el video del pitch (sección 7)	Rodrigo y Franco	Enlace de YouTube no listado probado
9	Revisar que ningún repositorio tenga secretos (comando abajo)	Franco	Sin resultados sensibles
10	Decidir quién envía cada formulario: Franco envía los pasos 3 y 4	Todos	Acordado

Publicar el demo

```
cd hcd
mkdir demo # copiar acá el prototipo: demo/index.html
git checkout -b feat/demo
git add demo
git commit -m "feat: add navigable prototype"
git push -u origin feat/demo # abrir pull request y merge a main

# Vercel (vercel.com)
# Add New > Project > importar el repo hcd
# Root Directory: demo Framework Preset: Other
# Deploy y copiar la URL
# Si la URL pide iniciar sesión: Settings > Deployment Protection > desactivar
# Plan B sin Git: app.netlify.com/drop y arrastrar la carpeta demo
```

Renombrar los repositorios a Salua (opcional hoy)

```
# En GitHub (solo el dueño): Settings > General > Repository name
# hcd -> salua, hcd_api -> salua_api, hcd_app -> salua_app, hcd_landing -> salua_landing
# Después, cada integrante en su computadora:
git remote set-url origin https://github.com/pmpeloc/salua.git
git remote -v
# Revisar variables de CI y de despliegue que mencionen el nombre anterior
```

Buscar secretos antes de hacer público un repo

```
git log --all -p | grep -iE "secret|private_key|api[_-]?key|mnemonic|BEGIN .*KEY" | head -20
# Si aparece una clave real: rotarla (crear una nueva). Borrarla del historial no alcanza.
```

Si algo falla con Vercel, Netlify Drop sirve como plan B porque no depende de Git ni de configuración.

Plantilla del README de hcd

```
# Salua
Tu historia clínica, bajo tu control.

Salua es una historia clínica digital donde el paciente decide quién la lee.
Los médicos verificados cargan estudios firmados; otros médicos solo los ven con
permiso del paciente, por tiempo limitado y con registro de cada acceso en Solana.
Los documentos nunca van a la cadena: se cifran en el navegador.

## Enlaces
- Demo navegable: <URL de Vercel>
- Pitch (deck y video): carpeta pitch/
- App: <repo hcd_app> - API y programa de Solana: <repo hcd_api>
- Página: <repo hcd_landing>

## Qué hay hoy y qué falta
- Hecho: plan técnico, prototipo navegable, diseño del programa. [ajustar al estado real]
- En construcción: programa en devnet, backend, servicio de llaves y app.

## Cómo funciona (resumen)
En la cadena: permisos con vencimiento, hashes y registro de accesos.
Fuera de la cadena: archivos cifrados y metadatos.

## Límites
No certificamos que un estudio sea clínicamente verdadero: certificamos quién lo emitió,
cuándo y que nadie lo alteró. El servicio de llaves es un punto de confianza.

## Equipo
Franco (líder técnico), Misael, Matías, Maximiliano y Rodrigo.
```

6. Entrega del domingo 4/10, paso a paso

Hora	Qué se hace	Quién
8:30	Llamada de 15 minutos: repasar el checklist y confirmar que los enlaces funcionan	Todos
9:00	Verificación cruzada: abrir demo, video, deck y repos en ventana de incógnito y con el celular con datos móviles	Rodrigo y Maximiliano
10:00	Completar los borradores de los pasos 3 y 4 con el kit de textos (anexo)	Franco
10:30	Congelar los materiales. Desde acá nadie edita demo, deck ni video	Todos
11:00	Enviar el paso 3 y guardar captura de la confirmación	Franco
12:00	Enviar el paso 4 (preselección) y guardar captura de la confirmación	Franco
12:30	Comprobar correos de confirmación y que Arena muestre el equipo completo	Rodrigo
13:00	Margen para corregir algo. Si todo está bien, no tocar nada más	Todos
16:00	Cierre de la preselección. No se envía nada después	—
16:30	Reunión técnica de 1 hora: congelar cuentas, contratos de API y la decisión sobre issue_record	Todos

Qué hacer en cada paso

Paso	Detalle
1 · Luma	Abrir el enlace y registrarse con el email del equipo. Sin esperar a último momento: son dos minutos
2 · Arena	Crear cuenta, crear el equipo o proyecto “Salua”, invitar a los cuatro restantes y confirmar que todos aceptaron. Si piden wallet, usar una wallet de Solana de cada persona
3 · Formulario de datos	Completar con los datos de cada integrante y del proyecto. Usar el kit de textos. Escribir lo que realmente existe hoy, sin exagerar el estado del código
4 · Preselección	Cargar enlaces del demo, del video, del deck y del repo hcd . Releer cada enlace antes de enviar. Es el formulario que decide la preselección

No digan que el programa está desplegado en devnet ni que el servicio de llaves funciona si todavía no es así. Lo que hoy existe, según el plan v2, es el plan técnico, el diseño del programa y un prototipo navegable sin backend. Digan eso, y presenten el recorrido completo como lo que van a construir hasta el 12/10. El jurado valora la honestidad, y una afirmación falsa se descubre al abrir el repositorio.

7. Grabar el pitch y subirlo

Tema	Cómo
Duración	2:30 a 3:00 para el video. Confirmar en el formulario si piden un máximo. Para presentar en vivo se usa el guion de 4:30
Cómo grabar	Grabar la pantalla con voz: Loom, OBS Studio, la barra de juegos de Windows (Win + G) o QuickTime en Mac. Probar el audio antes. Cerrar notificaciones
Qué decir	0:00 problema y frase de Salua (20 s). 0:20 a 2:20 demo del recorrido completo. 2:20 a 3:00 límites, siguiente paso y qué piden al programa
Dónde subirlo	YouTube con visibilidad “No listado”, o Google Drive con acceso “Cualquier persona con el enlace” como lector
Deck	Hacerlo en Google Slides, Canva o PowerPoint, exportarlo a PDF y guardarlo en <code>hcd/pitch/deck.pdf</code> . Las 8 diapositivas están en el plan completo, sección 4
Prueba	Abrir el enlace del video en una ventana de incógnito y desde el celular. Si pide permiso, está mal compartido

Prueba final antes de enviar (15 minutos)

- Cada enlace del formulario se abre en una ventana de incógnito sin iniciar sesión.
- El demo funciona en celular con datos móviles y muestra la marca Salua.
- El README de **hcd** se ve bien y cada enlace lleva al lugar correcto.
- No queda “HCD” visible en el demo, el deck, el video ni los repos que se enlazan.
- Ningún repositorio expone claves ni archivos .env.
- Las capturas de confirmación de los pasos 3 y 4 están guardadas en el grupo.

Si algo falla

Problema	Alternativa
Vercel no despliega o pide login	Netlify Drop con la carpeta demo, o desactivar la protección de despliegue
El video no sube	Subirlo a Google Drive con acceso por enlace y pegar ese enlace
No puedo hacer público un repo (no soy dueño)	Pedirle a quien administra pmpeloc que lo haga, o agregar al evaluador como colaborador si el formulario lo permite
El formulario no deja enviar	Probar otro navegador, guardar capturas del error y avisar al grupo. Escribir a Superteam Argentina antes de las 16:00
No llegamos con el deck	Enviar el README y el video, y entregar el deck apenas esté, si el formulario lo permite

Anexo · Kit de textos para pegar

Campo probable	Texto
Nombre	Salua
Frase	Tu historia clínica, bajo tu control.
Descripción corta	Salua es una historia clínica donde el paciente decide quién la lee: los médicos verificados cargan estudios firmados y otros médicos los ven solo con su permiso, por tiempo limitado y con registro en Solana. Los documentos nunca van a la cadena.
Problema	La historia clínica está repartida en cada clínica y el paciente no sabe quién la vio. La ley dice que el dueño es el paciente, pero en la práctica no lo es.
Solución	El paciente aprueba cada acceso, por tiempo limitado, y cada lectura queda registrada. Los estudios se cifran en el navegador y un servicio de llaves consulta el permiso en Solana antes de entregar cada llave.
Tecnología	Solana (Anchor), Next.js, NestJS, Supabase, Privy, Codama y cifrado AES-256-GCM con WebCrypto.
Por qué Solana	Permisos y auditoría verificables por cualquiera, con comisiones bajas y confirmación rápida, y un ecosistema de herramientas (Privy, Codama) que acelera el desarrollo.
Estado actual	Plan técnico completo, diseño del programa y prototipo navegable sin backend. El MVP funcional con el recorrido completo está en construcción para el 12/10. [Ajustar al estado real]
Qué construiremos hasta el 12/10	Programa en devnet con tests negativos, API con servicio de llaves, aplicación de paciente y médico con visor verificado y línea de tiempo de accesos.
Modelo de negocio (hipótesis)	Software para clínicas, prepagas u obras sociales, con cobro por organización. Estamos validando quién paga primero.
Límites	No certificamos que un estudio sea clínicamente verdadero, solo quién lo emitió, cuándo y que nadie lo alteró. El servicio de llaves es un punto de confianza que queremos reducir.
Equipo	Franco (líder técnico: arquitectura, servicio de llaves, cifrado e integración), Misael (programa Anchor y tests), Matías (backend), Maximiliano (frontend), Rodrigo (producto, validación, pitch y legal).
Contacto	[email y teléfono del líder]